

CloudPrintHub - Product Requirements Document (PRD)

1. Project Overview

Project Name

CloudPrintHub

Vision

Allow users to upload documents from anywhere in the world and print them remotely on their own computer-connected printer.

Primary Goal

Create a self-hosted cloud printing platform where:

- User uploads a document through a web portal.
- User selects printer settings.
- Django backend stores the print job.
- Windows Agent installed on a PC retrieves the print job.
- Agent sends document to the connected printer.
- User receives live status updates.

Target Audience

Phase 1

- Personal users
- Students
- Home office users

Phase 2

- Jan Seva Kendras
 - Cyber Cafes
 - Printing Shops
 - Small Businesses
-

2. Technical Stack

Backend

Framework

- Django 5+
- Django Templates
- Django ORM

Database

- SQLite (Development)
- PostgreSQL (Production)

File Storage

- Local Media Storage

Authentication

- Django Authentication System
-

Frontend

UI Technology

- Django Templates
- HTML5
- CSS3
- Bootstrap 5
- Vanilla JavaScript

No React for MVP.

Desktop Agent

Technology

Python 3.12+

Libraries

```
requests  
pywin32  
python-dotenv  
watchdog  
psutil
```

Supported OS

```
Windows 10  
Windows 11
```

3. System Architecture

High Level Flow

```
User  
↓  
Web Portal  
↓  
Django Backend  
↓  
Database  
↓  
Print Queue  
↓  
Windows Agent  
↓  
Connected Printer  
↓  
Printed Document
```

4. User Roles

Admin

Can:

- View all printers
 - View all jobs
 - Delete jobs
 - Disable printers
 - Monitor activity
-

User

Can:

- Login
 - Upload documents
 - Register printers
 - Create print jobs
 - View history
 - Monitor status
-

5. Core Features

Feature 1: User Authentication

Registration

Fields:

Name

Email

Password

Login

Fields:

Email
Password

Forgot Password

Email reset link.

Feature 2: Printer Registration

Purpose

Allow user to link a printer-connected PC.

Flow

```
Install Agent
↓
Login
↓
Register Device
↓
Generate Device Token
↓
Printer Appears In Dashboard
```

Printer Information

Store:

```
Printer Name
Computer Name
Windows Username
Device UUID
Last Seen
Status
```

Feature 3: Upload Document

Supported Files

Phase 1:

PDF

Phase 2:

PDF
DOCX
JPG
PNG

Upload Limits

Maximum File Size: 50MB

Feature 4: Print Job Creation

User Selects

Printer

Dropdown:

Home Printer
Office Printer
Shop Printer

Copies

Range:

1 - 999

Paper Size

Options:

A4
A3
Letter
Legal

Orientation

Options:

Portrait
Landscape

Color Mode

Options:

Color
Black & White

Duplex

Options:

Single Side
Double Side

Page Range

Examples:

All Pages
1-5
2,4,6

Quality

Options:

Draft
Normal
High

Feature 5: Print Queue

Statuses:

Pending
Downloaded
Printing
Completed
Failed
Cancelled

Feature 6: Print History

Display:

File Name
Printer
Date

Pages
Status

Feature 7: Printer Status

Statuses:

Online
Offline
Busy
Error

6. Database Design

User

id
name
email
password
created_at

Printer

id
user
name
device_uuid
token
computer_name
status
last_seen
created_at

Document

```
id
user
file
file_size
uploaded_at
```

PrintJob

```
id
user
printer
document

copies
paper_size
orientation
color_mode
duplex
page_range
quality

status

created_at
completed_at
```

7. API Design

Agent Authentication

POST

```
/api/agent/login/
```

Response:

```
{  
  "token": "abc123"  
}
```

Heartbeat

POST

```
/api/agent/heartbeat/
```

Sent every 60 seconds.

Fetch Jobs

GET

```
/api/agent/jobs/
```

Returns pending jobs.

Download Job

GET

```
/api/agent/job/{id}/download/
```

Downloads PDF.

Update Status

POST

```
/api/agent/job/{id}/status/
```

8. Windows Agent Architecture

Components

Agent Core

Responsible for:

- Authentication
 - Polling
 - Status updates
-

Printer Manager

Responsible for:

- Detecting printers
 - Reading capabilities
 - Managing print execution
-

Queue Processor

Responsible for:

- Processing jobs
 - Error handling
 - Retry logic
-

Logger

Responsible for:

- File logs
 - Error tracking
 - Diagnostics
-

9. Agent Workflow

```
Agent Starts
↓
Authenticate
↓
Heartbeat
↓
Check Queue
↓
Job Found
↓
Download File
↓
Validate File
↓
Print
↓
Update Status
↓
Wait For Next Job
```

10. Admin Dashboard

Sections

Dashboard

Display:

```
Users
Printers
Jobs
Completed Jobs
Failed Jobs
```

Printer Management

Actions:

View
Disable
Delete

Job Management

Actions:

View
Retry
Delete
Cancel

11. Security

Authentication

Use:

Django Session Authentication

Agent Security

Each printer receives:

Unique Token

Agent requests:

Authorization: Bearer TOKEN

File Validation

Check:

Extension
File Size
Mime Type

12. Hosting Plan

Development

Local Machine

MVP Deployment

Platform:

PythonAnywhere Free Tier

Database:

SQLite

Storage:

Local Media

13. Future Enhancements

Phase 2

Multi-Shop Printing

User chooses:

Nearest Shop

Print automatically.

Payments

Integrations:

Razorpay
Stripe

Mobile App

Technology:

React Native

Notifications

Email:

Job Completed
Job Failed

Real-Time Updates

Using:

Django Channels
WebSockets

14. Development Roadmap

Milestone 1

User System

Tasks:

- Authentication
 - Dashboard
 - User Profile
-

Milestone 2

Printer Registration

Tasks:

- Agent Authentication
 - Device Registration
 - Printer Discovery
-

Milestone 3

Document Upload

Tasks:

- Upload PDF
 - Storage
 - Validation
-

Milestone 4

Print Queue

Tasks:

- Create Jobs
 - Queue Processing
 - Status Tracking
-

Milestone 5

Windows Agent

Tasks:

- Polling
 - Downloading
 - Printing
 - Heartbeats
-

Milestone 6

Admin Dashboard

Tasks:

- Monitoring
 - Reporting
 - Management
-

15. MVP Success Criteria

A user should be able to:

1. Login.
2. Register a printer-connected PC.
3. Upload a PDF.
4. Select print settings.
5. Submit print job.
6. Have the Windows Agent automatically retrieve the job.
7. Print the document successfully.
8. View completion status in the dashboard.

When all eight actions work reliably, MVP is considered complete.